

ziVID

Small-Scale Pulp Deployment with Quadlets and NFS

PulpCon 2025 – 01 December 2025

Ricardo F. Guimarães



Why quadlets?

- Familiarity with **systemd**
- **Elegant** way to manage containers
- Easy access to **rootless podman** features
- **Simplicity** (when the number of nodes is small)

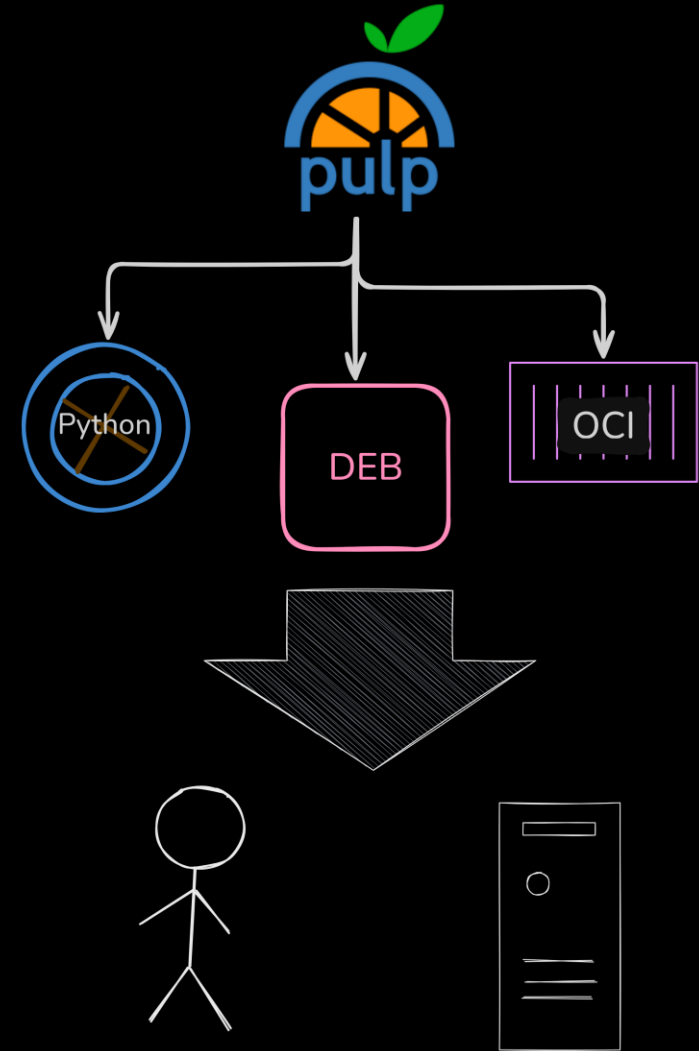


Agenda

- Context
- Deployment
- Conversion
- Issues
- Concluding remarks

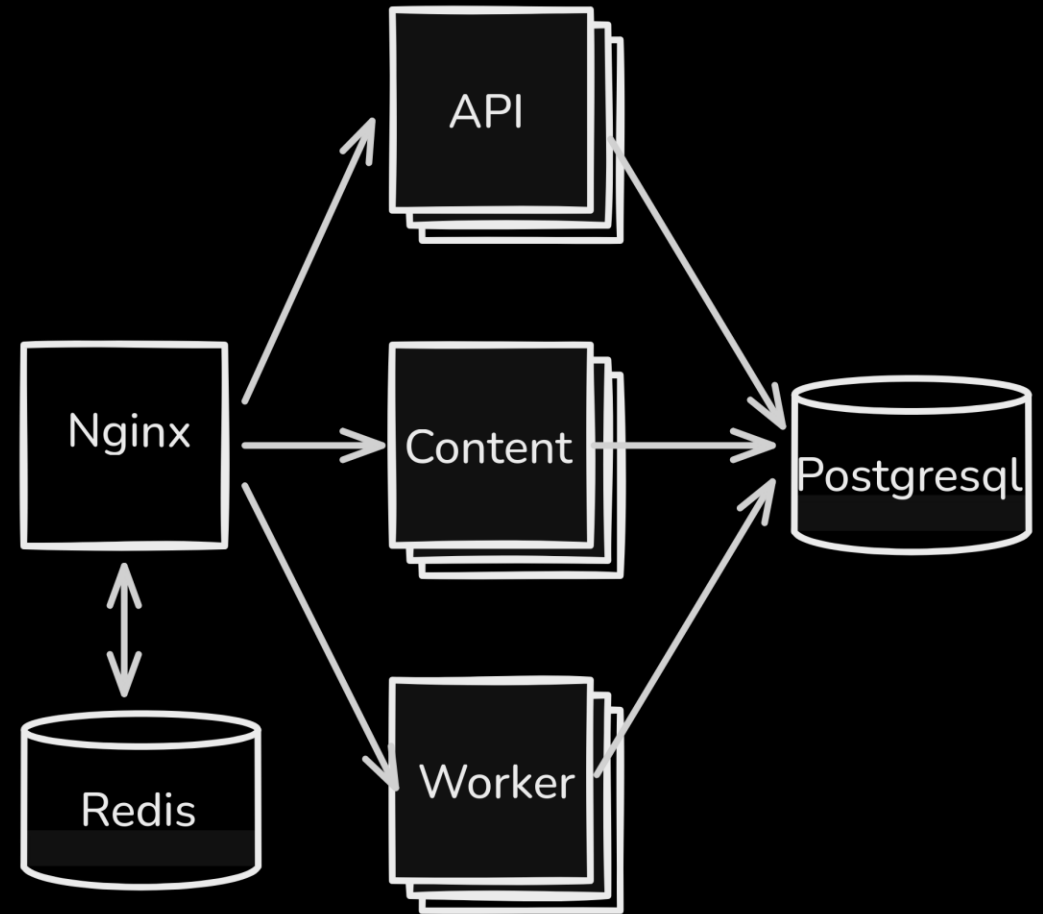
Pulp @ Zivid

- Industrial 3D cameras
 - ~ 50 engineers using infrastructure regularly
 - ~ 130 (quite busy) CI runners
- Tradition of self-hosted infrastructure
- Pulp:
 - Mirrors
 - Internal packages



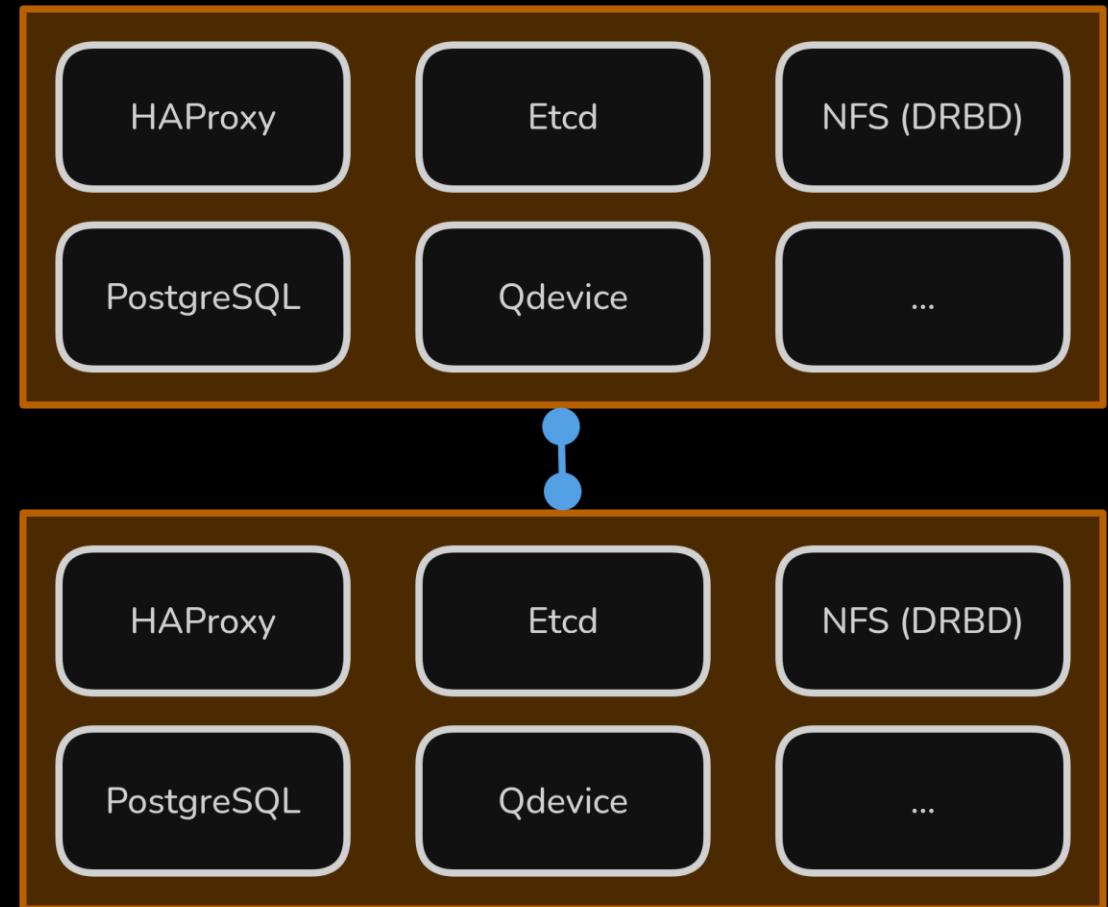
Initial Deployment

- Single-host
- Docker compose
- Deployed via bash scripts



From 1 to 2 nodes: why not compose or kubernetes?

- Compose:
 - Missing features that we need to manage permissions and hostnames
 - Requires **no_root_squash** or **chmod 777** on the NFS export
- Kubernetes: overkill for our use-case
- Quadlets:
 - Familiarity with systemd
 - Good middle ground
 - Easy to achieve a "safe" NFS setup



Quadlets

- Manage podman containers via systemd
- Thin, generator-based layer
- Services, network, volumes
- Declarative container configuration

```
# ~/.config/containers/systemd/hello-world.container
[Unit]
Description=Hello World one-shot container
After=network-online.target
Wants=network-online.target

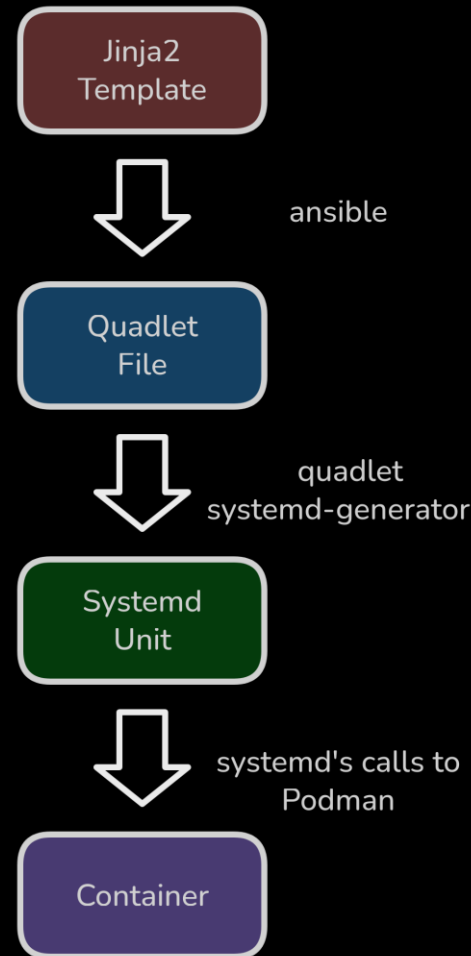
[Service]
Type=oneshot
Restart=no
TimeoutStartSec=30
SuccessExitStatus=0

[Container]
Image=hello-world
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

Deployment: From Ansible to Podman

- Infrastructure as Code
- Reduce manual work
- Work around some issues

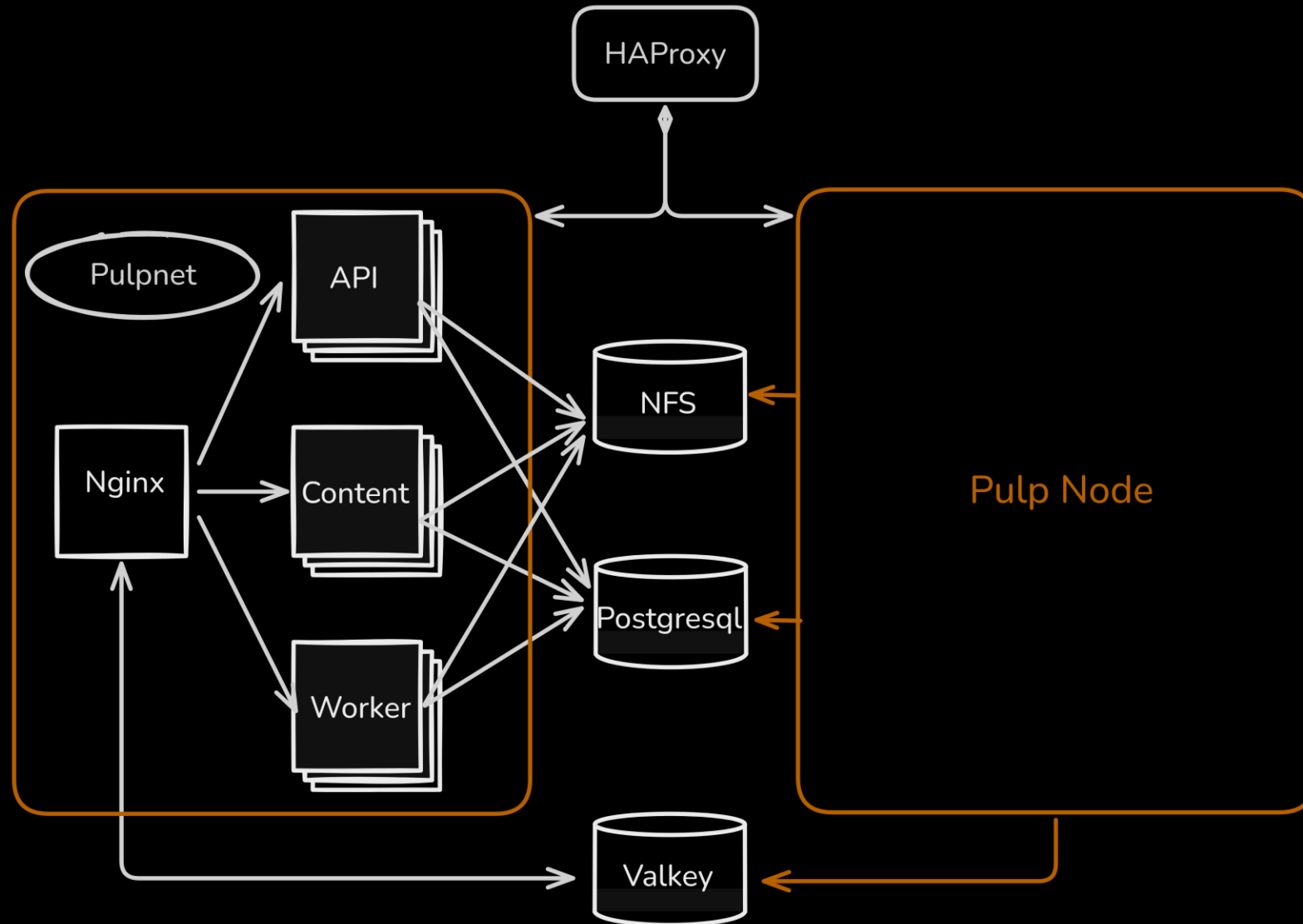


```
# In a terminal
mkdir -p ~/.config/containers/systemd
cp hello-world.container ~/.config/containers/systemd/
systemctl --user daemon-reload
systemctl --user start hello-world.service
systemctl --user status hello-world.service
```

```
● hello-world.service - Hello World one-shot container
   Loaded: loaded (/home/ci-admin/.config/containers/systemd/hello-world.service; vendor preset: enabled)
   Active: active (exited) since Mon 2025-12-01 08:49:03; 1min ago
 Invocation: 42689bf5a0cc409aba51b8a2dc909ef3
   Process: 2812078 ExecStart=/usr/bin/podman run --name hello-world --rm alpine:latest echo Hello World! (code=exited, status=0/SUCCESS)
  Main PID: 2812078
    Tasks: 0 (limit: 306188)
   Memory: 76K (peak: 54.9M)
      CPU: 236ms
   CGroup: /user.slice/user-1000.slice/user@1000.service


```

Overall architecture



Pulp container network via quadlets

```
# /home/pulp/.config/containers/systemd/pulpnet.network
[Unit]
Description=Pulp user-defined network
PartOf=pulp.target

[Network]
NetworkName=pulpnet
Driver=bridge
```

pulp-api@.container: 1/3

```
# docker-compose.yml
pulp_api:
  restart: always
  depends_on:
    set_init_password_service:
      condition: service_completed_successfully
  deploy:
    replicas: 6
    resources:
      limits:
        cpus: "2"
        memory: 6G
```

```
# pulp-api@.container
[Unit]
Description=Pulp API (%i)
PartOf=pulp.target
After=pulp-set-init-password.service
Requires=pulp-set-init-password.service

[Service]
Restart=on-failure
RestartSec=60s
# Resources
CPUQuota=200%
MemoryHigh=4G
MemoryMax=6G
MemorySwapMax=1G
```

pulp-api@.container: 2/3

```
# docker-compose.yml
  image: "pulp/pulp-minimal:3.69"
  command: ["pulp-api"]
  hostname: pulp-api
  user: pulp
```

```
# pulp-api@.container
[Container]
Image=quay.io/pulp/pulp-minimal:3.90
Exec=pulp-api
User=700
HostName=pulp-api-%i-se-pulp-1
StopTimeout=120
LogDriver=journald
LogOpt=tag={{.Name}}
UserNS=keep-id:uid=700,gid=700
Environment=PYTHONPATH=/opt/pulp
Environment=PGAPPNAME=pulp-api-%i-se-pulp-1

Network=pulpnet.network:alias=pulp_api_se-pulp-1
```

pulp-api@.container: 3/3

```
# docker-compose.yml
volumes:
  - "./settings/settings.py:/etc/pulp/settings.py:z"
  - "./assets/certs:/etc/pulp/certs:z"
  - "pulp_data:/var/lib/pulp:z"
healthcheck:
  test: readyz.py /pulp/api/v3/status/
  interval: 10s
  timeout: 5s
  retries: 5
```

```
# pulp-api@.container
HealthCmd=readyz.py /pulp/api/v3/status/
HealthInterval=10s
HealthRetries=5
HealthTimeout=5s

Volume=/home/pulp/pulp-server/settings/settings.py:/etc/pulp/settings.py:z
Volume=/home/pulp/pulp-server/assets/certs:/etc/pulp/certs:z
Volume=/home/pulp/pulp-server/settings/db_routing:/opt/pulp/pulp_router_ext:z
Volume=/mnt/pulp:/var/lib/pulp:z

[Install]
WantedBy=pulp.target
```

Systemd Target Example

```
# pulp.target
[Unit]
Description=Pulp stack (main target)

Wants=pulp-migration.service pulp-signing-key.service
Wants=pulp-set-init-password.service pulp-web.service
Wants=pulpnet-network.service
Wants=pulp-api@1.service pulp-api@2.service
Wants=pulp-content@1.service pulp-content@2.service
Wants=pulp-worker@1.service

After=network-online.target mnt-pulp.mount pulp-signing-key.service
After=pulp-network.service

Requires=mnt-pulp.mount pulp-web.service

[Install]
WantedBy=default.target
```

Avoiding name collisions: Pulp version

- `<hostname>@<pid>` was used for the database lock
- `<hostname>@<pid>` is still used on temporary directory names, and maybe elsewhere

[PULP-571] Drop session advisory locks from task execution #6923

 Merged mdellweg merged 5 commits into [pulp:main](#) from [mdellweg:571_task_locking](#) on Sep 8

3.89.0 (2025-09-09)

REST API

Features

- Stop using advisory locks in favor of a database column for locking on tasks.

Avoiding name collisions: templating

- Jinja2/Ansible: use VM hostname as part of the container's

```
# nginx.conf.template.j2
set $pulp_api pulp_api_{{ inventory_hostname }};
set $pulp_content pulp_content_{{ inventory_hostname }};
```

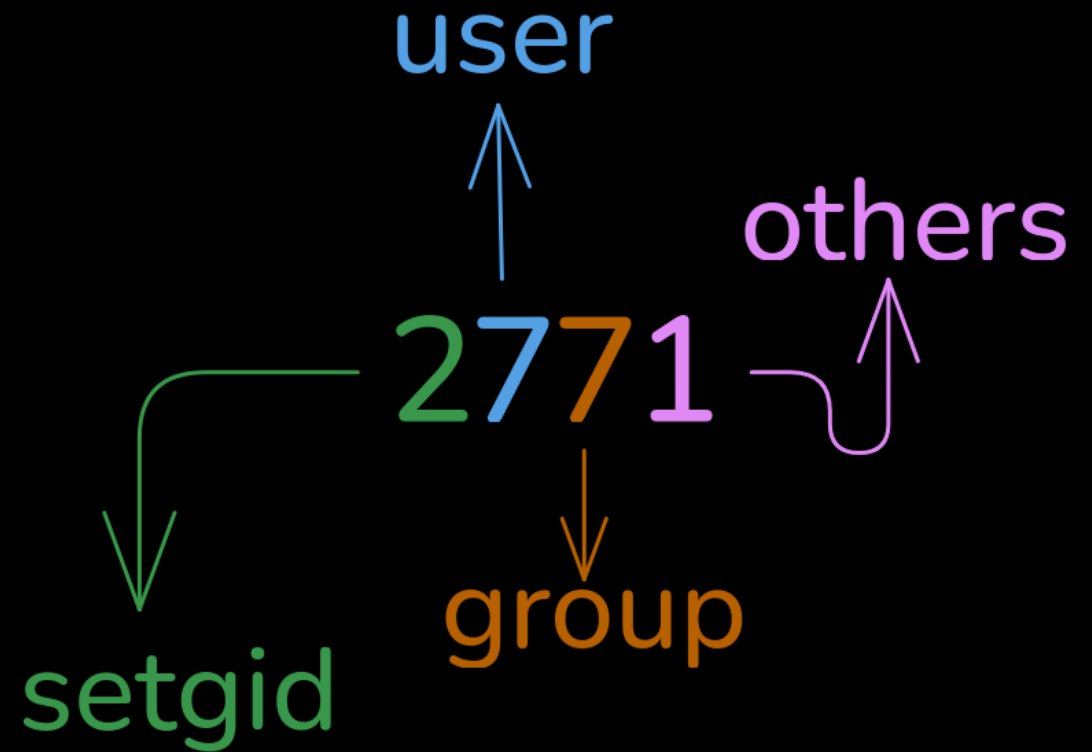
- Systemd unit file: use as templated unit (%i)

- Quadlet/Podman: alias the container to remove the index from the hostname

```
# pulp-api@.container.j2
HostName=pulp-api-%i-{{ inventory_hostname }} ...
Environment=PGAPPNAME=pulp-api_%i-{{ inventory_hostname }}
Network=pulpnet.network:alias=pulp_api_{{ inventory_hostname }}
```

File permissions and ownership

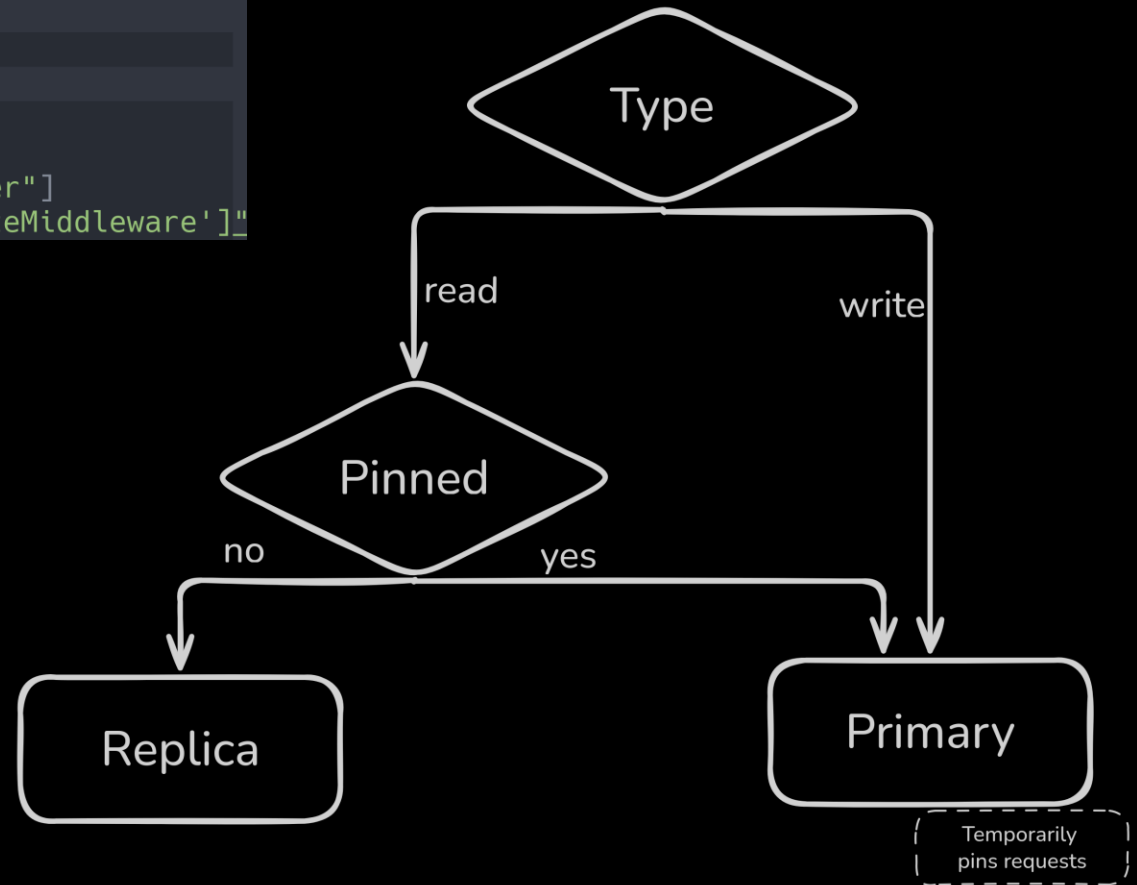
- Use the same PID/GID (700) everywhere
- Permission on the NFS export:
 - **u=rwx,g=rwx,o=x,g+s**
- Pulp containers on pulp user namespace
- Create these beforehand
 - .local/share/containers
 - media
 - tmp
 - scripts
- SE Linux for the NFS export:
 - **system_u:object_r:container_file_t:s0**



DB Routing

```
# settings.py
DATABASES = {
    "default": { ...
    },
    "replica": { ...
    },
}

DATABASE_ROUTERS = ["pulp_router_ext.dbrouters.PrimaryReplicaRouter"]
MIDDLEWARE = "@merge ['pulp_router_ext.middleware.PinPrimaryOnWriteMiddleware']"
```



Concluding remarks

- Quadlets are a viable solution for small, multi-node Pulp deployments
- It is generators (almost) all the way down
- Pulp nodes are quite independent
- Pulp is great! Thank you

Questions?



- Contact:

Ricardo (dot) Guimaraes (at) Zivid (dot)
com@rfguimaraes:matrix.org

The views and opinions expressed in this presentation are solely those of the author and do not necessarily reflect the views of Zivid.